Actividad de Pruebas Estáticas y Unitarias (Python, JavaScript y TypeScript)

Duración estimada: 45 minutos — Entorno: GitHub Codespaces

Descripción: La aplicación es un proyecto sencillo y multilenguaje (Python, JavaScript y TypeScript) que implementa funciones básicas —principalmente un verificador de palíndromos — con el objetivo de practicar pruebas estáticas y pruebas unitarias.

Cada versión del verificador funciona, pero contiene errores intencionales (como código inalcanzable, variables sin usar, comparaciones incorrectas o problemas de estilo) diseñados para que los estudiantes los identifiquen mediante herramientas de análisis estático.

El proyecto incluye también funciones simples adicionales (como suma en Python).

1. Objetivo de la actividad

Ejecutar análisis estático y escribir/aplicar pruebas unitarias en una aplicación con tres lenguajes (Python, JavaScript y TypeScript) dentro de GitHub Codespaces, sobre un proyecto funcional con *issues* intencionales que deben identificarse y corregirse.

2. Prerrequisitos

- Tener una cuenta de GitHub activa.
- Disponer de conexión a Internet para abrir el Codespace.
- Conocer comandos básicos de terminal (cd, npm, python).

3. Preparación del repositorio y Codespaces

1. Descarga el archivo ZIP del proyecto y guárdalo en tu equipo.

2. Crea un nuevo repositorio en tu cuenta de GitHub y sube el contenido del ZIP.

3. En GitHub, haz clic en Code → Codespaces → Create codespace on main.

4. Espera ~20–40 segundos. El contenedor instala automáticamente: Python (pytest, ruff), Node (npm, ESLint, Jest), TypeScript (tsc, Vitest).

Nota: No instales nada manualmente; el devcontainer trae todo preconfigurado.

4. Estructura del proyecto

```
multilanguage-quality-lab/
├─ README.md
├─ .devcontainer/devcontainer.json
├─ .vscode/extensions.json
├─ python_app/
│  ├─ palindromo.py
│  ├─ utils.py
│  ├─ requirements.txt
│  ├─ pytest.ini
│  └─ tests/test_lab.py
├─ js_app/
│  ├─ package.json
│  ├─ .eslintrc.json
│  ├─ src/stringUtils.js
│  └─ tests/stringUtils.test.js
└─ ts_app/
   ├─ package.json
   ├─ tsconfig.json
   ├─ .eslintrc.json
   ├─ src/stringUtils.ts
   └─ tests/stringUtils.test.ts
```

5. Ejecución en Python

Abre la terminal integrada y corre los siguientes comandos:

```
cd python_app
# Análisis estático (Ruff)
ruff check .
# Pruebas unitarias (pytest)
pytest -q
```

Tareas en Python (edita archivos dentro de python_app/):

• En tests/test_lab.py crea casos para es_palindromo: "radar"→True, "Radar"→True, "anita lava la tina"→True, "python"→False, cadena vacía → True.

• Agrega pruebas para utils.suma: (2,3)→5, (0,5)→5, (-2,3)→1.

• Corre de nuevo pytest y corrige los *issues* que reporte Ruff en palindromo.py (import sin uso, variable sin uso, código inalcanzable).

6. Ejecución en JavaScript (Node + Jest + ESLint)

```
cd ../js_app
# Análisis estático (ESLint)
npm run lint
# Pruebas unitarias (Jest)
npm test -- --watchAll=false
```

Tareas en JavaScript:

• Completa tests en tests/stringUtils.test.js para isPalindrome con los mismos casos usados en Python.

• Arregla *issues* típicos en src/stringUtils.js: usa === en lugar de ==, reemplaza var por const/let, elimina console.log de depuración y líneas inalcanzables.

7. Ejecución en TypeScript (tsc + Vitest + ESLint)

```
cd ../ts_app
# Lint para TS
npm run lint
# Type-check estricto (análisis estático semántico)
npm run typecheck
# Pruebas unitarias (Vitest)
npm test
```

Tareas en TypeScript:

• Completa tests en tests/stringUtils.test.ts para isPalindrome con los mismos casos (incluyendo mayúsculas/minúsculas y espacios).

• Corrige los *issues* marcados por ESLint y por el type-checker (—strict), y vuelve a ejecutar los comandos hasta quedar en verde.

8. Guía rápida de buenas prácticas (AAA y FIRST)

AAA: Arrange (prepara datos), Act (ejecuta la función), Assert (verifica el resultado).

FIRST: Fast, Independent, Repeatable, Self-Validating, Timely.

9. Criterios de logro

• Análisis estático: corregir ≥ 3 hallazgos por lenguaje (Python, JS, TS).

• Pruebas unitarias: Python (≥ 5 casos en palíndromo y ≥ 2 en suma); JS (≥ 4 casos en palíndromo); TS (≥ 4 casos en palíndromo).

• Ejecución: suites de pruebas verdes y linters/typecheck sin errores (warnings aceptables si se justifican).

10. Solución de problemas (FAQ rápida)

• El Codespace tarda o falla al iniciar: Cierra y vuelve a crear el Codespace. Verifica tu conexión. Si persiste, abre la pestaña "Ports" y la terminal para ver logs.

• ESLint o Ruff muestran muchos errores: Empieza por los de mayor severidad (error). Aplica cambios pequeños, guarda y vuelve a ejecutar el comando.

• Las pruebas no descubren mis archivos: Asegúrate de estar en la carpeta correcta y usa los comandos indicados. En Python, archiva pruebas como test_*.py; en JS/TS, revisa el patrón del runner.

• TypeScript marca errores de tipos: Lee el mensaje con atención. Ajusta el tipo o el código para cumplir las reglas del modo estricto. Vuelve a correr npm run typecheck.

11. Entrega sugerida

Realiza commits pequeños y descriptivos (por ejemplo: "test: agrega casos para palíndromo", "fix(lint): reemplaza == por ===''). Asegúrate de que las tres suites de pruebas y los linters pasen antes de finalizar.